Bandit (AST Pattern Matching)

Engine: Built-in Python ast (Node Visitor Pattern)

Language: Python Only

Work Level: Function Level / File Level (ตรวจแบบทีละบรรทัด/ทีละฟังก์ชัน ไม่มีการตาม Dataflow ข้ามฟังก์ชัน)

Detect: CWE, OWASP (เฉพาะ Web Flaws พื้นฐานของ Python) เช่น Hardcoded Passwords, Insecure Deserialization (Pickle), SQL Injection string, Shell Injection (subprocess)

Fix Support: ไม่มี Autofix (บอกแค่ชื่อ Rule ID และบรรทัด)

สรุปหลักการทำงานของ Bandit (How it Works)

Bandit เป็นเครื่องมือตรวจจับความปลอดภัยแบบ **Rule-Based (ใช้กฎเกณฑ์ตายตัว)** มันจะไม่รันโค้ดของคุณ และไม่ใช้ AI [3] แต่จะแปลงโค้ดภาษา Python ให้กลายเป็น โครงสร้างต้นไม้ที่เรียกว่า **AST (Abstract Syntax Tree)** จากนั้นจะส่ง **Plugins (ตัวสแกน)** วิ่งไล่เช็กตามกิ่งก้านของต้นไม้ เพื่อดูว่ามีพารามิเตอร์หรือฟังก์ชันไหนที่ตรงกับเงื่อนไขอันตรายที่ระบุไว้ในคู่มือหรือไม่ [3]

โค้ดเป้าหมายที่ Bandit กำลังจะส่อง

python

```python
def vulnerable_cwe_078_system_ping(host_address: str) -> str:
    command = f"ping -c 1 {host_address}"
    output = subprocess.check_output(command, shell=True)
    return output.decode("utf-8")
```

ใช้โค้ดอย่างระมัดระวัง

🌳 ขั้นที่ 1: แตกกิ่งก้านต้นไม้ (AST Parsing) จากบนลงล่าง

เมื่อ Bandit วิ่งเข้ามาในไฟล์นี้ มันจะสแกนตัวอักษรและแปลงรหัสโค้ดทั้งหมดให้กลายเป็นโครงสร้างต้นไม้ (Abstract Syntax Tree) โดยเรียงลำดับจากบนลงล่างแบบนี้:

text

```
[ Root: Module ]  (จุดเริ่มต้นของไฟล์)
   |
   └── [ Node 1: FunctionDef ] ➔ ชื่อฟังก์ชัน: "vulnerable_cwe_078_system_ping"
        |                        รับอาร์กิวเมนต์: "host_address"
        |
        ├── [ Node 2: Assign ] ➔ สร้างตัวแปรชื่อ: "command"
        |                        ค่าด้านใน: f-string (ผสมค่าว่า ping กับอินพุต)
        |
        ├── [ Node 3: Assign ] ➔ สร้างตัวแปรชื่อ: "output"
        |    └── [ Call ]      ➔ เรียกฟังก์ชัน: "subprocess.check_output" ⚠️
        |                        ส่งค่า: ตัวแปร "command" และ "shell=True"
        |
        └── [ Node 4: Return ] ➔ ส่งค่ากลับ: output.decode()
```

ใช้โค้ดอย่างระมัดระวัง

🚶 ขั้นที่ 2: วอล์กเกอร์ (Tree Walker) เริ่มเดินทัวร์ตรวจสอบทีละกิ่ง

หลังจากได้ต้นไม้มาแล้ว ตัวเดินเครื่องของ Bandit (Tree Walker) จะค่อยๆ เดินสายตรวจเช็กทีละข้อความ โดยส่งข้อมูลไปให้ **Plugins (บอทนักตรวจ)** ที่ลงทะเบียนรอไว้:

🛑 ก้าวที่ 1: ตรวจชื่อฟังก์ชัน `FunctionDef`

- **พฤติกรรม:** Walker เดินไปที่หัวฟังก์ชัน เห็นชื่อ `vulnerable_cwe_078_system_ping(host_address: str)`
- **สิ่งที่ Bandit ทำ:** มันมองผ่านกิ่งนี้ไปอย่างรวดเร็ว เนื่องจาก Bandit ไม่มีกฎ (Rule) ที่เขียนไว้ว่า *"ห้ามตั้งชื่อฟังก์ชันว่าอะไร"* หรือ *"ห้ามรับตัวแปรชื่ออะไร"* ดังนั้นในด่านนี้ **ไม่พบสิ่งผิดปกติ** โค้ดปล่อยผ่านฉลุยครับ

🛑 ก้าวที่ 2: ตรวจบรรทัดประกาศตัวแปร `Assign` (ตัวแปร command)

- **พฤติกรรม:** Walker ขยับลงมาเจอคำสั่งสร้างตัวแปร `command = f"ping..."`
- **สิ่งที่ Bandit ทำ:** มันตรวจสอบดูว่ามีการใส่รหัสผ่านลับ (Hardcoded Password) ลงในข้อความนี้ไหม (เช่น `password = "1234"`) แต่เนื่องจากข้อความนี้เป็นแค่คำสั่ง `ping` ธรรมดา บอทตรวจจับรหัสผ่านเลยเงียบ... ด่านนี้**ปล่อยผ่าน**

🚨 ก้าวที่ 3: ตรวจบรรทัดเรียกฟังก์ชัน `Call` (จุดเกิดเหตุ!)

- **พฤติกรรม:** Walker เดินมาเจอคำว่า `subprocess.check_output(command, shell=True)`
- **สิ่งที่ Bandit ทำ:** สัญญาณเตือนภัยดังขึ้น! เพราะ Bandit มีบอทที่ชื่อ `B602` ซึ่งทำหน้าที่เฝ้าระวัง `Call Node` (การเรียกฟังก์ชัน) ที่เกี่ยวกับระบบโดยเฉพาะ บอทตัวนี้จะกระโดดเข้ามาสแกนพารามิเตอร์ทันที:
  1. **เช็กย้อนกลับ (Traceback):** บอทหันไปมองว่าตัวแปร `command` ที่ถูกส่งเข้ามา มีที่มาจากไหน? พอมันย้อนไปดูก้าวที่ 2 มันพบว่าเป็น `f-string` (สตริงประกอบร่าง) ซึ่งแปลว่ามีอินพุตแปรผันผวนหลุดเข้ามาผสมแน่ๆ
  2. **เช็กคีย์เวิร์ดมรณะ:** บอทส่องตาเขม็งไปเจอคำว่า `shell=True`

📢 **ก้าวที่ 4: สรุปผลและปักธงแดง (Flagging)**

- **พอเงื่อนไขครบ (เป็นฟังก์ชัน `subprocess` + ใช้สตริงแบบ Dynamic + เปิด `shell=True`) บอท `B602` จะสั่ง หยุดเดิน ทันที และบันทึกตำแหน่งบรรทัดนี้ไว้เพื่อพ่น Warning ตัวเบ้อเร่อออกมาในรายงานความปลอดภัย**
- **ส่วนบรรทัด `return` ที่อยู่ถัดไป Bandit ก็จะเดินไปอ่านตามมารยาท แต่ไม่ได้สนใจอะไรเพิ่มเติม เพราะติดธงแดงที่จุดนี้ไปเรียบร้อยแล้วครับ**

(base) PS D:\4th-year\Senior-Project> bandit -r "Security&CWE/security_competitor_test_cases.py"

[main] INFO profile include tests: None

[main] INFO profile exclude tests: None

[main] INFO cli include tests: None

[main] INFO cli exclude tests: None

[main] INFO running on Python 3.14.7

Run started:2026-09-30 13:48:03.488194+00:00

Test results:

>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.

Severity: Low Confidence: High

CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle

Location: .\Security&CWE/security_competitor_test_cases.py:25:0

24 import json

25 import pickle

26 import sqlite3

>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.

Severity: Low Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess

Location: .\Security&CWE/security_competitor_test_cases.py:30:0

29 import tempfile

30 import subprocess

31 import requests

>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.

Severity: High Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html

Location: .\Security&CWE/security_competitor_test_cases.py:54:13

53 command = f"ping -n 1 {host_address}" if sys.platform == "win32" else f"ping -c 1 {host_address}"

54 output = subprocess.check_output(command, shell=True)

55 return output.decode("utf-8", errors="replace")

>> Issue: [B607:start_process_with_partial_path] Starting a process with a partial executable path

Severity: Low Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b607_start_process_with_partial_path.html

Location: .\Security&CWE/security_competitor_test_cases.py:66:13

65 ping_flag = "-n" if sys.platform == "win32" else "-c"

66 result = subprocess.run(

67 ["ping", ping_flag, "1", host_address],

68 capture_output=True,

69 text=True,

70 check=True,

71 timeout=5

72)

73 return result.stdout

>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.

Severity: Low Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subprocess_without_shell_equals_true.html

Location: .\Security&CWE/security_competitor_test_cases.py:66:13

65 ping_flag = "-n" if sys.platform == "win32" else "-c"

66 result = subprocess.run(

67 ["ping", ping_flag, "1", host_address],

68 capture_output=True,

69 text=True,

70 check=True,

71 timeout=5

72)

73 return result.stdout

>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.

Severity: Medium Confidence: Low

CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

Location: .\Security&CWE/security_competitor_test_cases.py:89:14

88 cursor = conn.cursor()

89 query = f"SELECT id, role, email FROM accounts WHERE username = '{username_input}' AND tenant_id = '{tenant_id}'"

90 cursor.execute(query)

>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.

Severity: Medium Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval

Location: .\Security&CWE/security_competitor_test_cases.py:138:11

137 """

138 return eval(user_expression)

139

>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue.

Severity: Medium Confidence: High

CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle

Location: .\Security&CWE/security_competitor_test_cases.py:295:11

294 """

295 return pickle.loads(serialized_cookie_data)

296

>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'super_secret_production_password_2026'

Severity: Low Confidence: Medium

CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html

Location: .\Security&CWE/security_competitor_test_cases.py:340:20

339 api_key = "sk_live_9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d"

340 client_secret = "super_secret_production_password_2026"

341 return {"api_key": api_key, "secret": client_secret}

>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False

Severity: High Confidence: High

CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html

Location: .\Security&CWE/security_competitor_test_cases.py:361:11

360 """

361 return hashlib.md5(password.encode("utf-8")).hexdigest()

362

>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.

Severity: High Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html

Location: .\Security&CWE/security_competitor_test_cases.py:469:14

```
468         llm_generated_command,
469         shell=True,
470         stdout=subprocess.PIPE,
471         stderr=subprocess.PIPE,
472         text=True
473     )
474     stdout, stderr = process.communicate()
475     return stdout if process.returncode == 0 else stderr
476
```

>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.

Severity: Low Confidence: High

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)

More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subprocess_without_shell_equals_true.html

Location: .\Security&CWE/security_competitor_test_cases.py:486:10

```
485         raise PermissionError(f"Unauthorized command binary: {parsed_tokens[0] if parsed_tokens else 'empty'}")
486     res = subprocess.run(parsed_tokens, capture_output=True, text=True, check=True, timeout=10)
487     return res.stdout
```

Code scanned:

Total lines of code: 414

Total lines skipped (#nosec): 0

Run metrics:

Total issues (by severity):

Undefined: 0

Low: 6

Medium: 3

High: 3

Total issues (by confidence):

Undefined: 0

Low: 1

Medium: 1

High: 10

Files skipped (0):

Semgrep OSS (CST / Tree-sitter)

Engine: CST (Concrete Syntax Tree) ผ่าน Tree-sitter + Intra-file Taint Tracking

Language: Multi-language (Python, JavaScript, TypeScript, Go, Java, C#, Ruby, PHP, Rust, C/C++, ฯลฯ)

Work Level: Function / File Level (ในรุ่น OSS) | Repository Level (Inter-file / Deep Taint ในรุ่น Pro/Commercial)

Detect: CWE, OWASP Top 10, Secrets/API Keys, และ CVE (เฉพาะรุ่นเสริม Semgrep Supply Chain / SCA ที่สแกน package.json, requirements.txt)

Fix Support: มี Autofix เบื้องต้นในกฎบางข้อ (ผ่านฟิลด์ autofix: แทนที่คำในบรรทัด)

โค้ดเป้าหมายที่เข้าสู่สายพานของ Semgrep

```python
def vulnerable_cwe_078_system_ping(host_address: str) -> str:
    command = f"ping -c 1 {host_address}"
    output = subprocess.check_output(command, shell=True)
    return output.decode("utf-8")
```

ใช้โค้ดอย่างระมัดระวัง

🔄 ขั้นตอนที่ 1: การเตรียมโครงสร้าง (File Parsing & CST Generation)

เมื่อคุณพิมพ์คำสั่งรันสแกน สิ่งแรกที่ Semgrep ทำคือการใช้เอนจิน **Tree-sitter** เปิดอ่านไฟล์และย่อยรหัสให้กลายเป็น **Concrete Syntax Tree (CST)**

- **เบื้องหลัง:** ต่างจาก Bandit ที่จะตัดคอมเมนต์ทิ้งไปทันที แต่ Semgrep จะเก็บโครงสร้างสภาพแวดล้อมโดยรอบ (Context) เอาไว้ครบถ้วน เพื่อให้ระบบสามารถวิเคราะห์ในระดับ Semantic (ความหมายเชิงลึก) ได้ โดยแปลงฟังก์ชันของคุณออกมาเป็นโหนด (Node) และกิ่งก้านสาขาในหน่วยความจำ

📂 ขั้นตอนที่ 2: โหลดกฎและปรับฟอร์แมต (Rule Loading & Normalization)

Semgrep จะดึงกฎที่คุณสั่งรัน (ในที่นี้คือ `--config auto` ซึ่งมีกฎ `python.lang.security.audit.subprocess-shell-true`) ขึ้นมาอ่าน

หน้าตาของกฎ (Rule Pattern) ที่สแกนเนอร์ใช้:

```yaml
pattern: subprocess.check_output($CMD, shell=True)
```

ใช้โค้ดอย่างระมัดระวัง

- **กระบวนการปรับมาตรฐาน (Normalization):** เอนจินของ Semgrep จะนำโครงสร้างโค้ดของคุณและโครงสร้างของกฎมาปรับฟอร์แมตให้เป็นมาตรฐานเดียวกันก่อน เพื่อเปิดใช้ฟีเจอร์ **Equivalence**
- เช่น ต่อให้คุณเขียนวงเล็บคร่อมแฝง หรือเขียนเว้นวรรคแปลกๆ ระบบจะถอดรูปให้เหลือเพียงเนื้อแท้ของคำสั่ง เพื่อป้องกันไม่ให้สไตล์การเขียนโค้ดที่ต่างกันมาหลอกสแกนเนอร์ได้

Step 1: ผลลัพธ์จาก Tree-sitter (Concrete Syntax Tree)

เมื่อ Tree-sitter อ่านบรรทัดนี้ มันจะแยกย่อยคำ (Tokenize) และโครงสร้างออกมาเป็น **CST** ซึ่งยังอิงตามชื่อ Node ใน Syntax ของภาษานั้น ๆ (เช่น มีเรื่องสัญลักษณ์อย่างเครื่องหมาย `=` และ `+`) แบบนี้ครับ:

text

```
program [0, 0] - [1, 0]
  expression_statement [0, 0] - [0, 9]
    assignment [0, 0] - [0, 9]
      left: identifier [0, 0] - [0, 1]  # "x"
      operator: '='
      right: binary_operator [0, 4] - [0, 9]
        left: integer [0, 4] - [0, 5]   # "1"
        operator: '+'
        right: integer [0, 8] - [0, 9]  # "2"
```

ใช้โค้ดอย่างระมัดระวัง

🚶 ขั้นตอนที่ 3: เดินท้วร์จับคู่พฤติกรรม (The Semantic Match Flow)

ตัวเดินเครื่อง (Walker) ของ Semgrep จะเริ่มวิ่งไปตามกิ่งก้านของต้นไม้โค้ดของคุณ จากบนลงล่าง:

1. เดินผ่านหัวฟังก์ชัน `vulnerable_cwe_078_system_ping`

- **ระบบคิดอย่างไร:** "นี่คือการประกาศฟังก์ชันใหม่ มีการรับพารามิเตอร์ชื่อ `host_address` บันทึกค่านี้ไว้ในตาราง Context ท้องถิ่น" (ยังไม่มีอะไรตรงกับกฎที่ระบุ)

2. เดินผ่านบรรทัดตัวแปร `command = f"ping..."`

- **ระบบคิดอย่างไร:** "มีการสร้างตัวแปรชื่อ `command` โดยเก็บสตริงที่เป็นประเภท f-string เอาไว้"

3. มาถึงจุดเกิดเหตุ: `output = subprocess.check_output(command, shell=True)`

ณ บรรทัดนี้ เอนจินการจับคู่ (Pattern Matching Engine) จะทำงานอย่างละเอียดแบบตัวต่อตัวกับกฎที่ตั้งไว้:

text

```
      [ โค้ดจริงของคุณ ]                    [ กฎ (Pattern) ของ Semgrep ]
subprocess.check_output(                 |  subprocess.check_output(
    command,                             |      $CMD,
    shell=True                           |      shell=True
)                                        |  )
                                         ▼
1. เช็กชื่อฟังก์ชัน ──► "subprocess.check_output" ตรงกับกฎเป๊ะ! (ผ่านด่านแรก)
2. เช็กพารามิเตอร์แรก ──► ในโค้ดคือ "command" ในกฎคือ "$CMD" (ตัวแปรสมมติ)
                          Semgrep สั่งแมปทันที: ให้ $CMD = ตัวแปร command (ผ่านด่านสอง)
3. เช็กคีย์เวิร์ดถัดไป ──► ในโค้ดมี "shell=True" ในกฎมี "shell=True" ตรงล็อก! (ผ่านด่านสาม)
```

ใช้โค้ดอย่างระมัดระวัง

🚨 ขั้นตอนที่ 4: ตรวจสอบความถูกต้องและสร้างผลลัพธ์ (Reporting & Output)

เมื่อการจับคู่ (Matching) ตรงเงื่อนไขครบทุกข้อ ตัวสแกนจะหยุดวิเคราะห์กิ่งนี้ และทำการแปลงข้อมูลทั้งหมดออกเป็นรายงานช่องโหว่ความปลอดภัย:

1. **ดึงพิกัด (Location):** ระบบดึงข้อมูลจาก CST ว่าจุดที่ตรงกับกฎอยู่ที่ บรรทัดที่ 54 คอลัมน์ที่ 13
2. **ดึงข้อความอธิบาย (Message):** นำคำอธิบายข้อผิดพลาดที่เขียนไว้ในชุดกฎ (Rule YAML) มาประกอบ
3. **พ่นผลลัพธ์ออกหน้าจอ:** แสดงแถบ `« Blocking »` พร้อมพ่นสัญลักษณ์ธงแดงว่าพบ `python.lang.security.audit.subprocess-shell-true.subprocess-shell-true` บน Terminal ของคุณทันทีครับ

```
  source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/
  b602_subprocess_popen_with_shell_equals_true.html
  owasp:
    - A01:2017 - Injection
    - A03:2021 - Injection
    - A05:2025 - Injection
  cwe:
    - "CWE-78: Improper Neutralization of Special Elements used in an OS
      Command ('OS Command Injection')"
  references:
    - https://stackoverflow.com/questions/3172470/actual-meaning-of-shell-true-in
    - https://docs.python.org/3/library/subprocess.html
  category: security
  technology:
    - python
  cwe2022-top25: true
  cwe2021-top25: true
```

(base) PS D:\4th-year\Senior-Project> C:\Users\Ongsa\miniconda3\Scripts\semgrep.exe scan --config auto "Security&CWE\security_competitor_test_cases.py"

┌──── ○○○ ────┐
│ Semgrep CLI │
└─────────────┘

Scanning 1 file (only git-tracked) with:

✓ Semgrep OSS

✓ Basic security coverage for first-party code vulnerabilities.

✗ Semgrep Code (SAST)

✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)

✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

💎 Get started with all Semgrep products via `semgrep login`.

✨ Learn more at https://sg.run/cloud.

━━ 100% 0:00:00

┌──────────────────┐
│ 9 Code Findings │
└──────────────────┘

Security&CWE\security_competitor_test_cases.py

❯❯❯ python.lang.security.audit.subprocess-shell-true.subprocess-shell-true

❮❮ Blocking ❯❯

Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command using a shell process. Doing so propagates current shell settings and variables, which makes it much easier for a malicious actor to execute commands. Use 'shell=False' instead.

Details: https://sg.run/J92w

▶▶┆ Autofix ▶ False

54┆ output = subprocess.check_output(command, shell=True)

❯❯ python.lang.security.audit.formatted-sql-query.formatted-sql-query

❮❮ Blocking ❯❯

Detected possible formatted SQL query. Use parameterized queries instead.

Details: https://sg.run/EkWw

90┆ cursor.execute(query)

❯❯❯ python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query

❮❮ Blocking ❯❯

Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.

Details: https://sg.run/2b1L

```
90 ┆ cursor.execute(query)
```

❯❯ python.lang.security.audit.eval-detected.eval-detected

❮❮ Blocking ❯❯

Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this content can be input from outside the program, this may be a code injection vulnerability. Ensure evaluated content is not definable by external sources.

Details: https://sg.run/ZvrD

```
138 ┆ return eval(user_expression)
```

❯❯ python.lang.security.deserialization.pickle.avoid-pickle

❮❮ Blocking ❯❯

Avoid using `pickle`, which is known to lead to code execution vulnerabilities. When unpickling, the serialized data could be manipulated to run arbitrary code. Instead, consider serializing the relevant data as JSON or a similar text-based serialization format.

Details: https://sg.run/OPwB

```
295 ┆ return pickle.loads(serialized_cookie_data)
```

❯❯❯ generic.secrets.security.detected-stripe-api-key.detected-stripe-api-key

❮❮ Blocking ❯❯

Stripe API Key detected

Details: https://sg.run/dKd5

339 ¦ api_key = "sk_live_9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d"

❯❯❯ generic.secrets.security.detected-picatic-api-key.detected-picatic-api-key

❮❮ Blocking ❯❯

Picatic API Key detected

Details: https://sg.run/rdGA

339 ¦ api_key = "sk_live_9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d"

❯❯ python.lang.security.audit.md5-used-as-password.md5-used-as-password

❮❮ Blocking ❯❯

It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by an attacker in a short amount of time. Use a suitable password hashing function such as scrypt. You can use `hashlib.scrypt`.

Details: https://sg.run/5DwD

361 ¦ return hashlib.md5(password.encode("utf-8")).hexdigest()

❯❯❯ python.lang.security.audit.subprocess-shell-true.subprocess-shell-true

❮❮ Blocking ❯❯

Found 'subprocess' function 'Popen' with 'shell=True'. This is dangerous because this call will spawn the command using a shell process. Doing so propagates current shell settings and variables, which makes it much easier for a malicious actor to execute commands. Use 'shell=False' instead.

Details: https://sg.run/J92w

▶▶ ¦ Autofix ▶ False

469 ¦ shell=True,

┌──────────────┐
│ Scan Summary │
└──────────────┘

✅ Scan completed successfully.

• Findings: 9 (9 blocking)

- Rules run: 290
- Targets scanned: 1
- Parsed lines: ~100.0%
- No ignore information available

Ran 290 rules on 1 file: 9 findings.

💎 Missed out on 1856 pro rules since you aren't logged in!

⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

ESLint (AST / Espree & Acorn)

Engine: AST Parser (Espree สำหรับ JS, @typescript-eslint/parser สำหรับ TS) เดินตรวจตาม Node Visitor

Language: JavaScript / TypeScript Only

Work Level: Line Level / Function Level / File Level (ตรวจทีละไฟล์เดี่ยวๆ ไม่มีการตาม Dataflow ข้ามไฟล์)

Detect: Code Quality, Syntactic Bugs, CWE (เฉพาะเมื่อติดตั้งปลั๊กอินเสริม เช่น eslint-plugin-security)

Fix Support: มี Autofix ผ่านคำสั่ง eslint --fix (แก้ได้เฉพาะ Formatting และ Syntax เล็กน้อย ไม่สามารถแก้ Security Flaws ได้)

🔄 ขั้นตอนที่ 1: การแกะโค้ดเป็น ESTree (Parsing Phase)

เมื่อคุณสั่งรันคำสั่งสแกน ESLint จะใช้เอนจินภายในที่ชื่อ **Espree** เพื่อแปลงซอร์สโค้ด JavaScript ของคุณให้กลายเป็น **AST (Abstract Syntax Tree)**

- **มาตรฐานเดียว:** ข้อดีของ Espree คือมันจะแปลงโค้ดให้อยู่ในรูปแบบมาตรฐานสากลของฝั่ง JavaScript ที่เรียกว่า **ESTree**
- ข้อดีของขั้นตอนนี้คือ มันจะมองข้ามวงเล็บส่วนเกิน มองข้ามช่องว่าง (Whitespace) และจัดโครงสร้างฟังก์ชัน ลำดับตัวแปร ให้กลายเป็นกิ่งก้านอย่างชัดเจน

📂 ขั้นตอนที่ 2: การโหลดระบบปลั๊กอินแบบ Event-Driven (Plugin Architecture)

นี่คือจุดเด่นที่สุดของ ESLint มันทำงานในรูปแบบ **Event-Driven (อิงตามเหตุการณ์ที่เจอ)**

1. ปลั๊กอิน `eslint-plugin-security` จะทำการลงทะเบียน (Subscribe) ไว้กับ ESLint ว่า *"ถ้าตัววอล์กเกอร์เดินไปเจอโหนดประเภทไหน ให้เรียกฉันด้วยนะ"*
2. ตัวอย่างเช่น กฎ `security/detect-child-process` จะขอดักฟังเหตุการณ์เมื่อเจอ `CallExpression` (การเรียกใช้ฟังก์ชัน)
3. กฎ `security/detect-object-injection` จะขอดักฟังเหตุการณ์เมื่อเจอ `MemberExpression` (การเข้าถึง Property ของ Object เช่น `obj[key]`)

🚶 ขั้นตอนที่ 3: เดินทัวร์และแกะรอยทีละฟังก์ชัน (The Execution Flow)

เมื่อตัว Walker เริ่มเดินไล่ตรวจเช็กโหนดของแต่ละฟังก์ชันจากบนลงล่าง นี่คือสิ่งที่เกิดขึ้นเบื้องหลังในแต่ละจุดที่มันพ่น Warning ออกมาครับ:

🚨 จุดที่ 1: บรรทัดที่ 27 (CWE-78: `security/detect-child-process`)

```javascript
exec(`tar -czf backup.tar.gz ${userFileName}`, (err, stdout, stderr) => { ... })
```

ใช้โค้ดอย่างระมัดระวัง

- **โครงสร้าง AST ที่เจอ:** `CallExpression` ที่มีการเรียกฟังก์ชันชื่อ `exec`
- **การคิดของปลั๊กอิน:**
  - กฎตรวจสอบว่าชื่อฟังก์ชันที่ถูกเรียกคือ `exec` ซึ่งอยู่ในลิสต์เฝ้าระวังของคลัง `child_process`
  - มันเข้าไปส่องอาร์กิวเมนต์ตัวแรก (พารามิเตอร์แรก)
  - มันพบว่าอาร์กิวเมนต์นี้ไม่ใช่ข้อความตายตัว (Literal) แต่เป็น `TemplateLiteral` (สตริงที่มีการใช้รูปแบบ `${...}` เพื่อเอาตัวแปรมาแปะ)
  - **ผลลัพธ์:** เข้าเงื่อนไข *"เรียกคำสั่งระบบโดยใช้อินพุตที่แปรผันได้"* ปลั๊กอินจึงยกธงแจ้งเตือนบรรทัด 27:3 ทันที

🚨 จุดที่ 2: บรรทัดที่ 51 (CWE-1321: `security/detect-object-injection`)

```javascript
targetConfig[key] = parsed[key];
```

ใช้โค้ดอย่างระมัดระวัง

- **โครงสร้าง AST ที่เจอ:** พบโหนดประเภท `MemberExpression` ถึง 2 จุดในบรรทัดเดียว คือ `targetConfig[key]` และ `parsed[key]`
- **การคิดของปลั๊กอิน:**
  - กฎตัวนี้ถูกตั้งไว้ว่า *"ถ้ามีการเข้าถึงข้อมูลใน Object ผ่าน วงเล็บเหลี่ยม `[]` โดยค่าข้างในวงเล็บไม่ใช่สตริงตายตัว (Non-Literal) ให้ตีเป็นความเสี่ยงทั้งหมด"*
  - ในโค้ดของคุณ ตัวข้างในคือตัวแปร `[key]` ซึ่งเปลี่ยนค่าได้ตลอดเวลาตามลูป `for...in`
  - **ผลลัพธ์:** มันพ่นแจ้งเตือนออกมา 2 รอบซ้อน (`51:5` และ `51:25`) สำหรับตัวแปรทั้งสองฝั่ง
  - *หมายเหตุ (ข้อจำกัด):* จุดนี้เป็นจุดที่สร้าง **False Positive (แจ้งเตือนมั่ว)** สูงมากใน ESLint เพราะต่อให้เราเขียนโค้ดปกติตามมาตรฐาน JavaScript เพื่อวนลูปเช็กข้อมูลธรรมดาที่ไม่มีช่องโหว่ ปลั๊กอินนี้ก็จะบ่นอยู่ดีครับ

🚨 **จุดที่ 3: บรรทัดที่ 65 (CWE-022: `security/detect-non-literal-fs-filename`)**

```javascript
return fs.readFileSync(filePath, 'utf-8');
```

ใช้โค้ดอย่างระมัดระวัง

- **โครงสร้าง AST ที่เจอ:** `CallExpression` ของฟังก์ชัน `fs.readFileSync`
- **การคิดของปลั๊กอิน:** ปลั๊กอินจะวิ่งเข้าไปเช็กพารามิเตอร์ช่องแรกทันที มันพบว่าตัวแปร `filePath` เป็นตัวแปรประเภท `Identifier` (มาจากตัวแปรที่คำนวณข้างบน ไม่ใช่ข้อความตายตัว อย่าง `"./safe.txt"`) มันจึงแจ้งเตือนทันทีเพราะกลัวว่าตัวแปรนี้จะถูกแทรกแซงเพื่อรัน Path Traversal

🚨 **จุดที่ 4: บรรทัดที่ 78 (OWASP LLM05 & CWE-94: `security/detect-eval-with-expression`)**

```javascript
const result = eval(action.generatedScript);
```

ใช้โค้ดอย่างระมัดระวัง

- **โครงสร้าง AST ที่เจอ:** `CallExpression` รันคำสั่งระดับตำนานอย่าง `eval()`
- **การคิดของปลั๊กอิน:** ตัวสแกนพบคำว่า `eval` และเมื่อส่องอาร์กิวเมนต์ด้านในก็พบโครงสร้างแบบ `MemberExpression` (`action.generatedScript`) ซึ่งเข้าตำราคำสั่งอันตรายที่มีการส่งผ่านค่าตัวแปรดั้งเดิม จึงถูกปักธงแจ้งเตือนไปตามระเบียบครับ

🚨 จุดสังเกตสำคัญสำหรับเล่ม Senior Project (The AI Agent Blindspot)

หากสังเกตในฟังก์ชันที่ 4 `vulnerable_owasp_llm05_agent_eval_action` โค้ดนี้จำลองสถานการณ์ยุคปัจจุบันที่ **AI Agent เป็นผู้สร้างสคริปต์มาทำ Action แล้วส่งเข้าลูป `eval()`**

- **ข้อจำกัดของ ESLint:** มันมองเห็นแค่ทางกายภาพว่านี่คือคำสั่ง `eval` ทั่วไป (แจ้งเตือนผ่านกฎพื้นฐาน) แต่ตัวมันไม่มีบริบท **(Context Blindspot)** เลยว่านี่คือสถาปัตยกรรมของ AI Agent และไม่สามารถประเมินภัยคุกคามรูปแบบใหม่อย่าง **Indirect Prompt Injection** ได้ ซึ่งตรงนี้จะเป็นจุดขายที่ดีมากให้กับโปรเจกต์ของคุณ (`CodeLens AI / AgentShield`) ในการชู้โรงว่าระบบของคุณเข้าใจบริบทในเชิงลึกมากกว่าครับ

D:\4th-year\Senior-Project\Security&CWE\security_competitor_test_cases.js

27:3 warning Found child_process.exec() with non Literal first argument security/detect-child-process

51:5 warning Generic Object Injection Sink security/detect-object-injection

51:25 warning Generic Object Injection Sink security/detect-object-injection

65:10 warning Found readFileSync from package "fs" with non literal argument at index 0 security/detect-non-literal-fs-filename

78:18 warning eval with argument of type MemberExpression security/detect-eval-with-expression

✖ 5 problems (0 errors, 5 warnings)

Snyk Code (DeepCode AI / Symbolic Taint Graph)

Engine: Symbolic AI + Machine Learning Cloud Taint Graph

Language: Multi-language กว้างมาก (JavaScript/TypeScript, Python, Java/Kotlin, C#/.NET, Go, PHP, Ruby, Rust, Scala, Swift/Objective C, Dart/Flutter, Apex, Groovy, C/C++, COBOL)

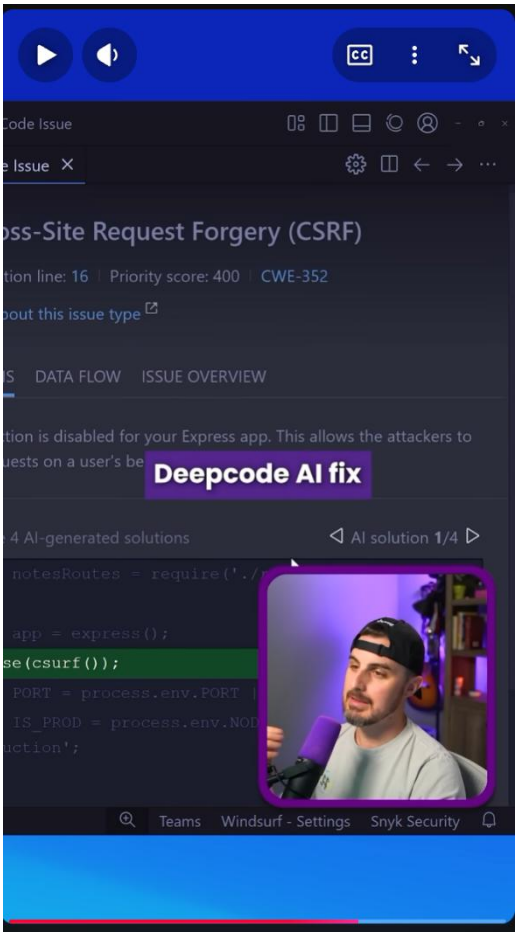
Work Level: Repository Level (Inter-procedural & Cross-File บน Cloud)

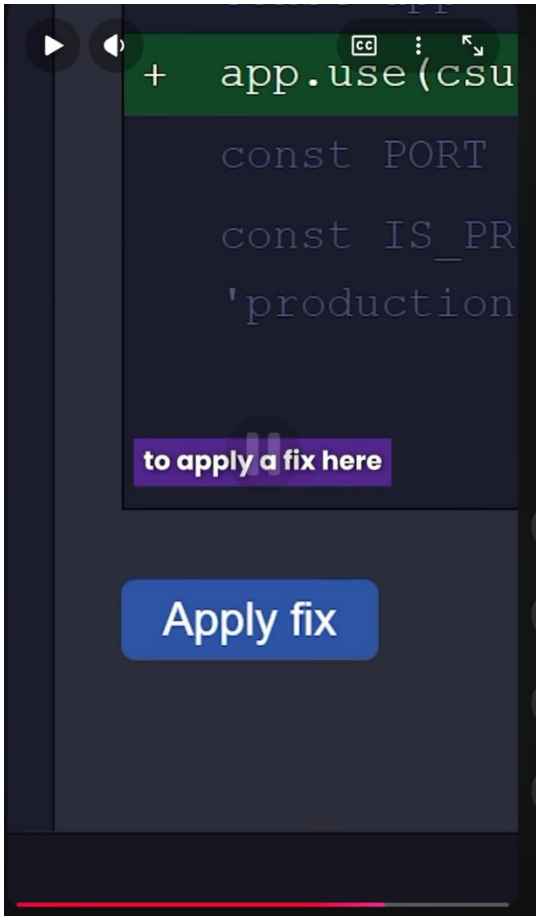
Detect: CWE, OWASP Top 10, และ CVE (เฉพาะใน Snyk Open Source / SCA ที่สแกน Dependency Manifest)

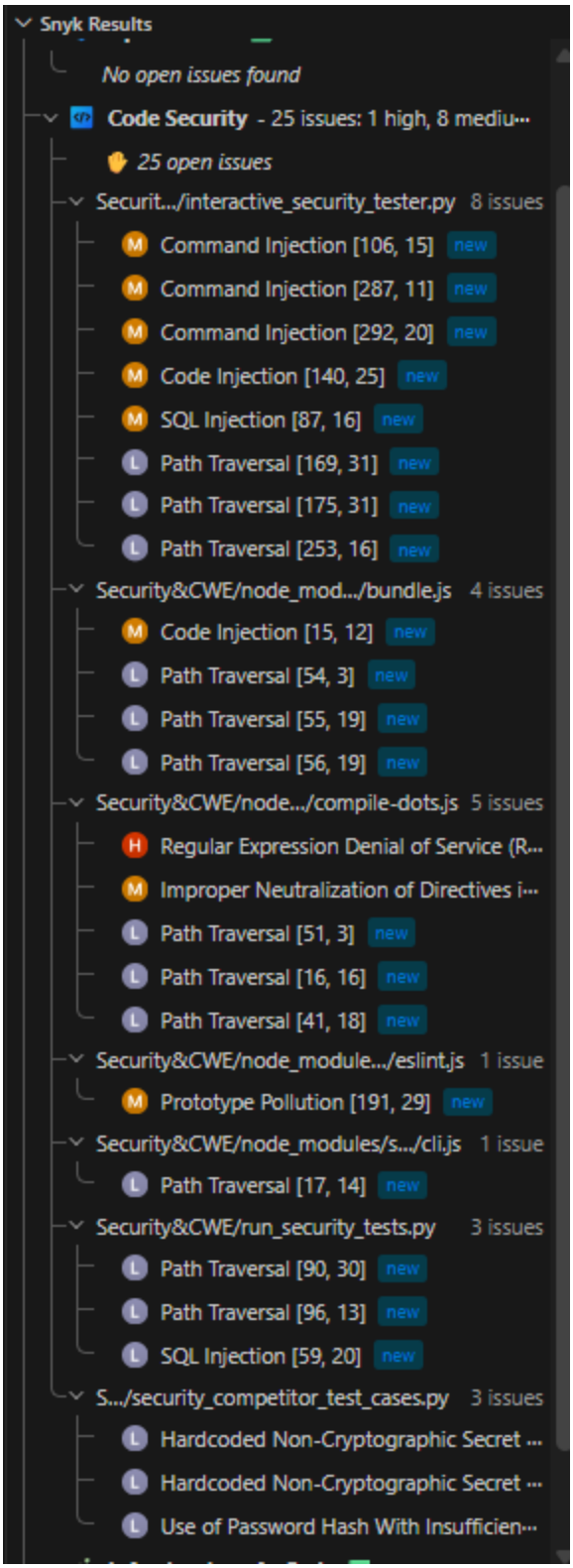
Fix Support: มี DeepCode AI Suggestions (Snyk Fix) ช่วยแนะแนวทางแก้โค้ด (แต่จำกัดการใช้งานจริง ต้องจ่ายเงิน Enterprise Tier)

ตารางเปรียบเทียบราคา Snyk Code (DeepCode AI)

แผน (Plan)	ราคาเริ่มต้น	ขีดจำกัดการทดสอบและฟีเจอร์เด่น
Free (ฟรี)	**$0**	จำกัด **100 tests / เดือน** สำหรับ Snyk Code (มีปลั๊กอิน IDE)
Team	**$25 / นักพัฒนา / เดือน**	จำกัด **1,000 tests / เดือน** (จำกัดทีมนักพัฒนา 5-10 คน)
Ignite	**$1,260 / นักพัฒนา / ปี**	**ไม่จำกัดจำนวนครั้งการทดสอบ (Unlimited)** สำหรับทีมขนาด 11-50 คน
Enterprise	**ราคาแบบปรับแต่ง (Custom)**	**ไม่จำกัดจำนวนครั้ง** + ฟีเจอร์การกำกับดูแลและความปลอดภัยขั้นสูง


Code Issue
Issue
oss-Site Request Forgery (CSRF)
tion line: 16 | Priority score: 400 | CWE-352
bout this issue type
S DATA FLOW ISSUE OVERVIEW
tion is disabled for your Express app. This allows the attackers to
uests on a user's be
Deepcode AI fix
4 AI-generated solutions
AI solution 1/4
notesRoutes = require('./
app = express();
se(csurf());
PORT = process.env.PORT
IS_PROD = process.env.NOD
uction';
Teams
Windsurf - Settings
Snyk Security


+ app.use(csu
const PORT
const IS_PR
'production
to apply a fix here
Apply fix


Snyk Results
No open issues found
Code Security - 25 issues: 1 high, 8 mediu...
25 open issues
Securit.../interactive_security_tester.py 8 issues
Command Injection [106, 15] new
Command Injection [287, 11] new
Command Injection [292, 20] new
Code Injection [140, 25] new
SQL Injection [87, 16] new
Path Traversal [169, 31] new
Path Traversal [175, 31] new
Path Traversal [253, 16] new
Security&CWE/node_mod.../bundle.js 4 issues
Code Injection [15, 12] new
Path Traversal [54, 3] new
Path Traversal [55, 19] new
Path Traversal [56, 19] new
Security&CWE/node.../compile-dots.js 5 issues
Regular Expression Denial of Service (R...
Improper Neutralization of Directives i...
Path Traversal [51, 3] new
Path Traversal [16, 16] new
Path Traversal [41, 18] new
Security&CWE/node_module.../eslint.js 1 issue
Prototype Pollution [191, 29] new
Security&CWE/node_modules/s.../cli.js 1 issue
Path Traversal [17, 14] new
Security&CWE/run_security_tests.py 3 issues
Path Traversal [90, 30] new
Path Traversal [96, 13] new
SQL Injection [59, 20] new
S.../security_competitor_test_cases.py 3 issues
Hardcoded Non-Cryptographic Secret ...
Hardcoded Non-Cryptographic Secret ...
Use of Password Hash With Insufficien...

⚠ Medium Severity | SQL Injection | Priority Score 353 | Vulnerability: CWE-89

Unsanitized input from user input flows into execute, where it is used in an SQL query. This may result in an SQL Injection vulnerability.

Details

In an SQL injection attack, the user can submit an SQL query directly to the database, gaining access without providing appropriate credentials. Attackers can then view, export, modify, and delete confidential information; change passwords and other authentication information; and possibly gain access to other systems within the network. This is one of the most commonly exploited categories of vulnerability, but can largely be avoided through good coding practices.

Best practices for prevention

- Avoid passing user-entered parameters directly to the SQL server.
- Avoid using string concatenation to build SQL queries from user-entered parameters.

Best practices for prevention

- Avoid passing user-entered parameters directly to the SQL server.
- Avoid using string concatenation to build SQL queries from user-entered parameters.
- When coding, define SQL code first, then pass in parameters. Use prepared statements with parameterized queries. Examples include `SqlCommand()` in .NET and `bindParam()` in PHP.
- Use strong typing for all parameters so unexpected user data will be rejected.
- Where direct user input cannot be avoided for performance reasons, validate input against a very strict allowlist of permitted characters, avoiding special characters such as `? & / < > ; - ' " \` and spaces. Use a vendor-supplied escaping routine if possible.
- Develop your application in an environment and/or using libraries that provide protection against SQL injection.
- Harden your entire environment around a least-privilege model, ideally with isolated accounts with privileges only for particular tasks.

ลองสร้างฟังก์ชั่นที่ต้องเรียกกันข้ามไฟล์แต่มี Vulnerability อยากรู้มันจะตรวจเจอมั้ย เอาแบบในโค้ดปลอดภัยแต่เรียกข้ามกันแล้วอันตราย

3) Snyk Code (DeepCode AI)

powershell

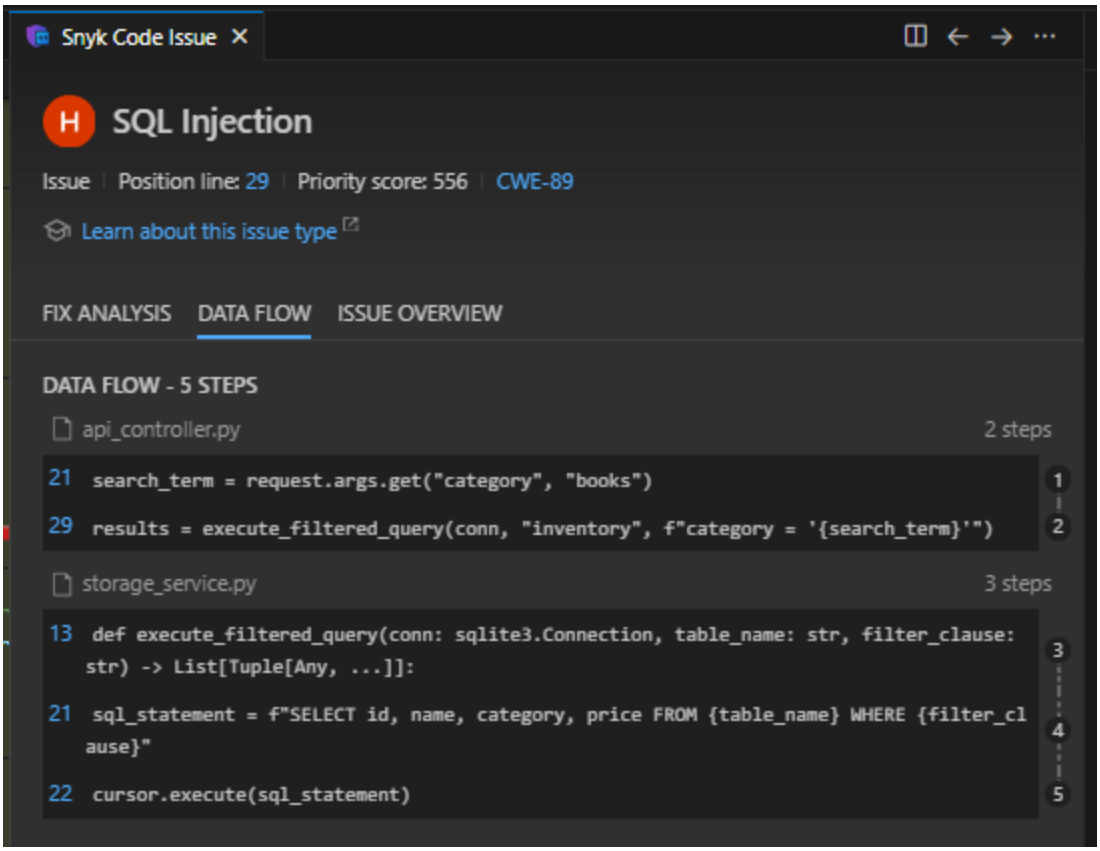
```
snyk code test "Security&CWE\cross_file_demo"
```

- **ผลลัพธ์:**

text

```
Testing Security&CWE\cross_file_demo ...
Open Issues:
  X [HIGH] SQL Injection
    Path: api_controller.py, line 29
    Info: Unsanitized input from an HTTP parameter flows into execute, where it is used
in an SQL query.
```

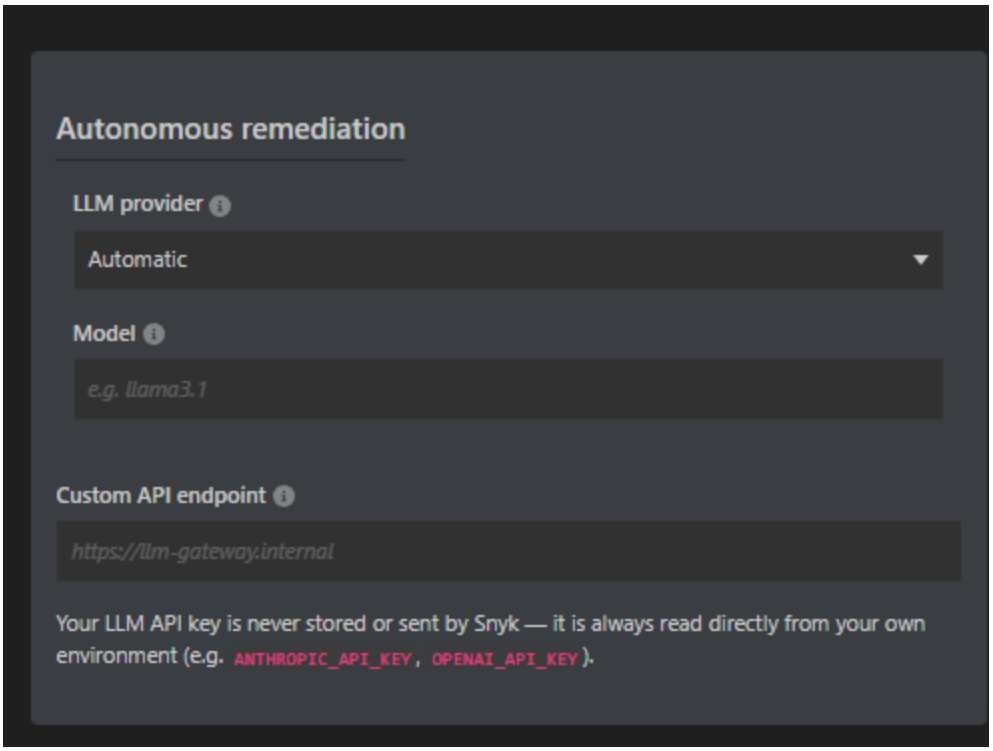
- **Snyk Code ตรวจเจอ!** เพราะ Snyk Code มีความสามารถด้าน **Inter-procedural Taint Analysis** มันสามารถตามรอยจาก `request.args` ใน `api_controller.py` ข้าม `import` ทะลุเข้าไปถึง `cursor.execute` ใน `storage_service.py` ได้!



2. ขอบเขตการติดตั้งใช้งาน (Where it Operates)

Snyk ถูกออกแบบมาให้เป็น **Developer-First Security Platform** มันจึงสามารถนำไปปล๊กอินและทำงานได้ทุกที่ในวัฏจักรการพัฒนาซอฟต์แวร์ (SDLC) เหมือนกับ SonarQube ครับ:

- **Local Machine / IDE Level (ระดับที่คุณรันอยู่ในรูป):** ทำงานบนเครื่องคอมพิวเตอร์ของคุณผ่าน VS Code Extension โดยมันจะแอบจับตาดูไฟล์ที่คุณกำลังเขียนและโฟลเดอร์โปรเจกต์ (`Directory`) เพื่อแปลงเป็นกราฟความสัมพันธ์ก่อนส่งไปประมวลผล
- **Repository / GitHub Level:** คุณสามารถเชื่อมต่อ Snyk เข้ากับ GitHub, GitLab หรือ Bitbucket ได้โดยตรง มันจะทำหน้าที่เฝ้า `Repository` ทั้งอัน เมื่อมีนักพัฒนาสร้าง **Pull Request (PR)** สแกนเนอร์ของ Snyk จะทำการสแกนโค้ดส่วนที่เพิ่มเข้ามาใหม่ทันที หากเจอช่องโหว่มันจะทำตัวเป็นบอทคอยคอมเมนต์แจ้งเตือนในไลน์โค้ดบน GitHub ให้เลย
- **CI/CD Pipeline Level (เหมือน SonarQube):** สามารถนำไปใส่ใน Jenkins, GitHub Actions, GitLab CI หรือ CircleCI โดยตั้งค่าให้เป็นด่านตรวจ (Quality Gate) หากในโปรเจกต์มีแฮกเกอร์แอบส่งโค้ดที่มีช่องโหว่ระดับ High ติดเข้ามา Snyk จะสั่ง **Fail Build** เพื่อบล็อกไม่ให้โค้ดชุดนั้นหลุดขึ้นเซิร์ฟเวอร์จริง (Production)



Allow custom endpoint like Azure but don’t allow local model

เจาะลึกการทำงานในแต่ละสเต็ป (Step-by-Step Breakdown)

Step 1: Parsing & Tokenization (การแกะโครงสร้างโค้ด)

- **การทำงาน:** ทันทีที่คุณทำการ Commit โค้ด หรือกดสั่งสแกนผ่าน IDE, Snyk Code จะนำไฟล์โค้ดภาษาต่างๆ (เช่น JavaScript, Python, Java) มาตัดย่อยและแปลงสภาพให้กลายเป็น **Abstract Syntax Tree (AST)**
- **เป้าหมาย:** สกัดเอา "หน้าตาไวยากรณ์ภายนอก" ออกไปให้เหลือเฉพาะ "ตรรกะและโครงสร้างที่แท้จริง" ของโค้ด เพื่อให้ระบบคอมพิวเตอร์เข้าใจลำดับการทำงาน

Step 2: Graph Construction (การสร้างแผนผังความสัมพันธ์)

- **การทำงาน:** Snyk Code จะยกระดับจาก AST ปกติ โดยการสร้างสิ่งที่เรียกว่า **Event Graph (EG)** แผนผังนี้จะประกอบไปด้วย:
  - **Control Flow Graph:** โค้ดวิ่งไปทางไหน (เช่น เส้นทางเงื่อนไข `if/else`, การวนลูป `loop`)
  - **Data Flow Graph:** ตัวแปรไหนส่งค่าต่อไปให้ตัวแปรไหนบ้าง
- **เป้าหมาย:** ทำความเข้าใจพฤติกรรมของโค้ดแบบข้ามไฟล์ (Cross-file) และข้ามฟังก์ชัน (Inter-procedural) ทั่วทั้งโปรเจกต์

Step 3: Taint Analysis (การตามรอยข้อมูลปนเปื้อน)

- **การทำงาน:** ระบบจะทำหน้าที่เป็นสายสืบ คอยตามรอยข้อมูลจาก 2 จุดสำคัญ:
  - **Source (ต้นตอ):** จุดที่รับข้อมูลจากภายนอกเข้ามา (เช่น `request.body`, ช่องกรอกฟอร์มของผู้ใช้)
  - **Sink (ปลายทาง):** จุดที่ข้อมูลนั้นถูกนำไปประมวลผลอันตราย (เช่น คำสั่งรันคำสั่ง SQL `db.execute()`, คำสั่งเรนเดอร์หน้าเว็บ)
- **เป้าหมาย:** ตรวจดูว่ามีข้อมูลจาก **Source** ไหลไปถึง **Sink** โดยที่ไม่มีการตัดคำสั่งอันตรายออก (Sanitization) หรือไม่

Step 4: AI & Symbolic Solver Evaluation (การตัดสินด้วยสมองกล)

- **การทำงาน:** นี่คือหัวใจสำคัญของ DeepCode AI ระบบจะนำแผนผังปัญหาที่กรองได้ไปวิ่งผ่าน **Symbolic AI (Logic Solver)** ควบคู่กับข้อมูลฐานความรู้ที่เรียนรู้มาจาก Open-source นับล้านโปรเจกต์ (Machine Learning)
- **เป้าหมาย:** แยกแยะเจตนาของโค้ด (Semantic Intent) เพื่อลดอัตราความผิดพลาด (False Positives) เช่น หากโค้ดบรรทัดนั้นดูเหมือนพัง แต่จริงๆ มีการครอบระบบป้องกันที่หนาแน่นไว้ก่อนหน้าแล้ว AI ก็จะกรองออก ไม่แจ้งเตือนให้คุณรำคาญ

Step 5: Output Generation (การส่งรายงานผล)

- **การทำงาน:** ส่งผลลัพธ์กลับมาที่หน้าจอผู้ใช้ โดยแมปผลลัพธ์เข้ากับ **CWE ID** อย่างเจาะจง พร้อมแสดงจุดบกพร่องเป็นเส้นทางเดินของโค้ด (Data Flow Path) บรรทัดต่อบรรทัดเพื่อให้คุณเข้าใจง่ายที่สุด

CodeQL (Relational Code DB & Datalog) (AST + CFG + DFG)

Multiple languages: C/C++, C#, Go, Java/Kotlin, JavaScript/TypeScript, Python, Ruby, Swift

Work on Repository Level (Whole-Program Analysis)

Detect: CWE, OWASP, Code Quality & Bug Patterns, CVE (GitHub Dependabot)

Have Copilot Autofix suggestions (GitHub Advanced Security / Paid Enterprise only)

